ТЕХНИЧЕСКОЕ ЗАДАНИЕ

TeamMind — интеллектуальный помощник для формирования рабочих команд

Тип: кроссплатформенное desktop-приложение для управления проектными командами.

Цель: анализировать загрузку, отпуска, навыки и рабочие предпочтения сотрудников, чтобы предлагать сбалансированные команды.

Компонент	Технологии и назначение
Frontend	React + TypeScript + Vite — интерфейс и клиентская логика
Desktop	Electron — настольная оболочка и сборка приложения
Backend	Python + Django REST Framework — бизнес-логика и REST API
GraphQL	Graphene-Django или Strawberry — гибкие запросы и мутации данных
База данных	PostgreSQL — постоянное хранение данных
Дополнительно	JWT, Recharts, Docker Compose, pytest, Git

Документ для командной разработки в рамках учебного проекта колледжа.

1. Общая информация

Заказчик — организация, сотрудники которой объединяются в группы для выполнения проектов.

Исполнитель разрабатывает desktop-приложение для управления сотрудниками, проектами, отпусками и автоматического формирования рабочих команд.

Основной результат: руководитель задаёт требования проекта, система анализирует доступность сотрудников и предлагает несколько подходящих составов с оценкой и объяснением решения.

2. Роли пользователей

Администратор

• Создаёт учётные записи, назначает роли и управляет отделами и справочниками.

• Настраивает права доступа и просматривает журнал важных действий.

Руководитель проекта

• Создаёт проекты и задачи, задаёт сроки, необходимые навыки и число участников.

• Просматривает загрузку и отпуска, запускает алгоритм формирования команд.

• Сравнивает варианты, редактирует состав и утверждает команду.

Сотрудник

• Обновляет доступные поля профиля, навыки и рабочие предпочтения.

• Указывает хронотип и предпочтительные часы встреч.

• Подаёт сведения об отпуске и просматривает свои назначения.

3. Функциональные требования

3.1. Авторизация и доступ

• Вход по логину и паролю, выход из системы и безопасное хранение паролей.

• Разграничение функций по ролям с обязательной проверкой разрешений на backend.

• Защита REST и GraphQL API от неавторизованных запросов.

• Создание учётных записей администратором; восстановление пароля — дополнительная функция.

3.2. Управление сотрудниками

Карточка сотрудника содержит ФИО, должность, отдел, навыки и уровни навыков, доступные рабочие часы в неделю, текущую загрузку, хронотип, предпочтительный интервал работы и статус доступности. Реализовать поиск, фильтрацию, сортировку, создание, редактирование и архивирование.

3.3. Календарь отпусков

• Просмотр отпусков по сотрудникам и отделам.

• Создание и редактирование периодов; статусы: ожидает подтверждения, одобрен, отклонён.

• Предупреждение об отсутствии необходимых специалистов в период проекта.

• Одобренный отпуск, пересекающийся с проектом, исключает сотрудника из доступных кандидатов

на этот период.

3.4. Проекты и задачи

Проект содержит название, описание, даты начала и завершения, число участников, необходимые навыки и минимальные уровни компетенций, предполагаемую трудоёмкость, приоритет и предпочтительное время совместной работы. Внутри проекта создаются задачи и назначения сотрудников.

4. Анализ загрузки и рабочих предпочтений

4.1. Загрузка сотрудников

Загрузка рассчитывается как занятые рабочие часы / доступные рабочие часы × 100%. Система показывает процент занятости, выявляет перегрузку, ищет свободных специалистов, сравнивает отделы и прогнозирует нагрузку после назначения на проект.

4.2. Хронотипы

Сотрудник может выбрать «жаворонок», «сова» или «промежуточный тип» и указать предпочтительные часы встреч. Алгоритм учитывает пересечение доступных интервалов.
Хронотип
используется как рабочее предпочтение, а не как показатель профессиональной пригодности.

4.3. Совместимость

• Совпадение предпочтительных часов работы.

• Предпочтения по коммуникации и командной/самостоятельной работе.

• Взаимодополняющие навыки и добровольно указанные предпочтения взаимодействия.

• Выявление конфликтующих расписаний.

Система не ставит психологические диагнозы и не выдаёт субъективную совместимость за научно доказанный факт. Результат является рекомендацией, которую руководитель может проверить и изменить.

5. Алгоритм формирования команд

• Исключить сотрудников, недоступных в период проекта.

• Проверить обязательные навыки и минимальные уровни компетенций.

• Оценить свободную загрузку кандидатов.

• Учесть совместимость расписаний и рабочие предпочтения.

• Сформировать несколько вариантов, если это возможно.

• Рассчитать общий балл и объяснить причины выбора.

• Разрешить ручное изменение состава и утверждение руководителем.

Критерий

Вес

Соответствие профессиональным навыкам

35%

Доступность и отсутствие отпусков

25%

Баланс рабочей нагрузки

20%

Совместимость расписаний

10%

Предпочтения взаимодействия

10%

Итого

100%

S = 0,35K + 0,25A + 0,20L + 0,10T + 0,10C. K — соответствие навыкам, A — доступность, L — баланс нагрузки, T — расписания, C — предпочтения. Показатели нормируются от 0 до 100.

Обязательные ограничения проверяются отдельно: высокий общий балл не компенсирует отсутствие необходимого специалиста или отпуск. Для сложной оптимизации можно использовать OR-Tools.

Результат должен показывать покрытие навыков, прогноз загрузки, общие интервалы встреч, риски и причины, по которым альтернативный состав лучше или хуже. Если подходящую команду найти нельзя, система сообщает, каких ресурсов не хватает.

6. Аналитическая панель

Показывает количество сотрудников и активных проектов, сотрудников в отпуске, среднюю загрузку, перегруженных сотрудников, проекты без команды, дефицит специалистов, загрузку отделов, распределение по хронотипам и последние запуски алгоритма.

7. Архитектура и интеграции

Компонент

Назначение

Electron

Desktop-оболочка; отображает React-интерфейс и обеспечивает сборку.

React + TypeScript

Страницы, формы, таблицы, календарь, графики и навигация.

Django REST Framework

REST API для стандартных операций, авторизации и интеграций.

GraphQL

Гибкое получение связанных данных и мутации через Graphene-Django или Strawberry.

PostgreSQL

Постоянное хранение пользователей, сотрудников, проектов и результатов.

Выполнение длительных операций: генерация команд, отчёты, импорт/экспорт и уведомления.

7.1. Использование GraphQL

• Создать GraphQL endpoint, например /graphql/.

• Реализовать Query для сотрудников, проектов, отпусков, загрузки и вариантов команд.

• Реализовать Mutation для создания/редактирования сотрудников, проектов, отпусков и утверждения команды.

• Проверять аутентификацию и права доступа для каждого запроса и поля, где это необходимо.

• Ограничить глубину и сложность запросов; использовать пагинацию и защиту от чрезмерной выборки.

• Не дублировать без причины все REST endpoints: REST оставить для стандартных CRUD/интеграционных сценариев, GraphQL — для гибких связанных выборок и сложных экранов.

Примеры операций: запрос карточки проекта вместе с участниками и требуемыми навыками; получение загрузки сотрудников отдела; мутация запуска генерации команды.

• Генерация команды, построение больших отчётов и импорт таблиц выполняются асинхронно, чтобы не блокировать интерфейс и HTTP-запрос.

• API возвращает идентификатор задачи и статус: queued, running, succeeded или failed.

• Frontend периодически запрашивает статус через API либо получает обновления через WebSocket/SSE как дополнительную функцию.

• Обрабатывать повторные попытки, тайм-ауты и ошибки; задачи по возможности делать идемпотентными.

долговременное хранилище бизнес-данных.

PostgreSQL → клиент получает статус и запрашивает результат.

7.3. Безопасность Electron

Electron является клиентской оболочкой, Django — отдельным backend-сервисом. Не размещать

секретные ключи, пароли базы данных или административные полномочия внутри клиентского приложения. Использовать безопасную конфигурацию окна, contextIsolation и отключённый nodeIntegration для renderer-процесса, если он не нужен.

8. Структура PostgreSQL

Таблица

Назначение

users

Учётные записи и роли.

employees

Профили сотрудников.

departments / positions

Отделы и должности.

skills / employee_skills

Справочник навыков и уровни сотрудников.

work_preferences

Хронотип и рабочие предпочтения.

vacations

Периоды отпусков и статусы.

projects / tasks

Проекты и задачи.

task_assignments

Назначения сотрудников на задачи.

team_runs

Запуски алгоритма и их статусы.

teams / team_members

Сформированные команды и участники.

compatibility_preferences

Добровольные предпочтения взаимодействия.

audit_logs

Журнал важных действий.

Использовать первичные и внешние ключи, ограничения целостности, индексы для частых запросов, миграции Django и историю изменений состава команд.

9. API и контракты

Интерфейс

Пример

Назначение

REST

POST /api/auth/login/

Авторизация

REST

GET /api/employees/

Список сотрудников

REST

GET /api/projects/

Список проектов

REST

POST /api/projects/{id}/generate-team/

REST

GET /api/tasks/{task_id}/

GraphQL

POST /graphql/

Query и Mutation для связанных данных

GraphQL

teamOptions(projectId: ID!)

Запрос вариантов команд

GraphQL

approveTeam(teamId: ID!)

Мутация утверждения команды

API-контракты должны включать форматы запросов и ответов, ошибки, права доступа, фильтрацию

и пагинацию. Для GraphQL необходимо документировать схему типов, Query, Mutation и правила

авторизации.

10. Требования к интерфейсу

• Авторизация.

• Главная аналитическая панель.

• Список сотрудников и карточка сотрудника.

• Календарь отпусков.

• Список проектов, карточка проекта и задачи.

• Конструктор команд и сравнение вариантов.

• История команд, управление пользователями и настройки.

Для страниц предусмотреть состояния загрузки, ошибки API, пустые списки, подтверждение

11. Нефункциональные требования

• Безопасность: права на backend, безопасное хранение паролей и валидация данных.

• Производительность: пагинация; тестирование набора минимум из 100 сотрудников.

• Поддерживаемость: модульная структура, единый стиль кода и документация.

• Прозрачность: сохранять критерии, статусы и объяснения распределения.

• Конфиденциальность: собирать только необходимые данные; ограничивать доступ к предпочтениям.

• Переносимость: сборка приложения для выбранных настольных ОС.

12. Этапы разработки

Этап

Результат

1. Проектирование

Макеты, ER-диаграмма, архитектура, REST/GraphQL контракты.

2. Backend и БД

Django, PostgreSQL, авторизация, роли и CRUD.

3. Frontend

React-страницы, формы, календарь и графики.

4. Алгоритм

Оценка кандидатов, ограничения, варианты и объяснения.

6. Electron

Desktop-оболочка и сборка.

7. Тестирование

8. Сдача

Демо-данные, инструкция запуска и презентация.

13. Критерии приёмки

• Пользователь может войти, а система различает роли.

• Администратор управляет сотрудниками, отделами и навыками.

• Руководитель создаёт проекты и задачи.

• Отпуска и доступность учитываются при подборе кандидатов.

• Загрузка рассчитывается автоматически.

• Алгоритм учитывает навыки, нагрузку, расписания и обязательные ограничения.

• Для вариантов отображаются баллы, объяснения и предупреждения.

• Руководитель может изменить состав и утвердить команду.

• Данные сохраняются в PostgreSQL, аналитика отображает реальные данные.

• REST API и GraphQL работают с проверкой прав доступа.

• React-приложение запускается через Electron.

• Есть документация, автоматические тесты и инструкция запуска.

14. Что передать заказчику

• Исходный код React + TypeScript, Electron и Django.

• Схему базы данных и миграции.

• Документацию REST API и GraphQL-схемы.

• Инструкцию установки и запуска.

• Демонстрационные данные и автоматические тесты.

• Презентацию и демонстрацию полного сценария.

15. Дополнительные функции

• Сравнение вариантов команды и What-if-анализ.

• Поиск дефицита компетенций по активным проектам.

• Импорт сотрудников и отпусков из CSV/XLSX.

• Экспорт отчётов в PDF и Excel.

• Уведомления о перегрузке и конфликтах расписаний.

• История изменений состава команды.

• Оптимизация нескольких проектов одновременно.

Рекомендуемый объём: сначала полностью реализовать основной поток — сотрудники → навыки →

→ утверждение. GraphQL использовать для связанных запросов интерфейса. Дополнительные функции добавлять после стабилизации основного сценария.

Конец технического задания